

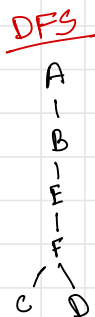
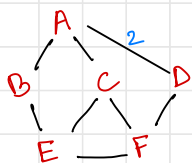
DS II Final

Sayeen



Final

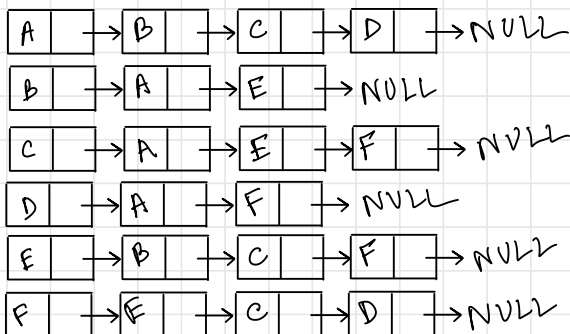
- BFS, DFS
- Disjoint Sets
- Minimum Spanning Tree
- Shortest Path
- Hash Table
- String Match (Rabin Karp)
- NP Complete X



Adjacency Matrix:

	A	B	C	D	E	F
A	0	1	1	2	0	0
B	1	0	0	0	1	0
C	1	0	0	0	1	1
D	1	0	0	0	0	1
E	0	1	1	0	0	1
F	0	0	1	1	1	0

Adjacency List:



Applications for BFS/DFS:

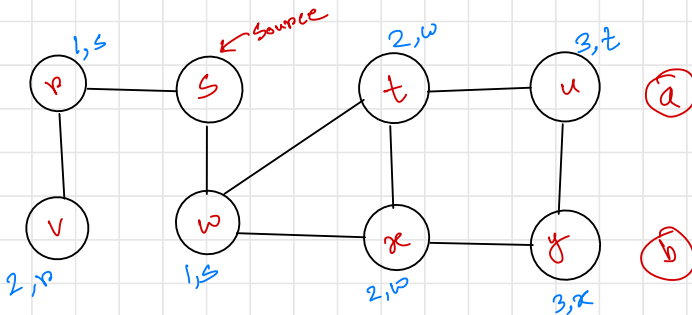
BFS

- Path Finding
- Shortest Path (Unweighted Tree)
- Connected Component
- Cycle
- Bipartite

DFS

- Path Finding
- Cycle
- Strongly Connected
- Topological Sort

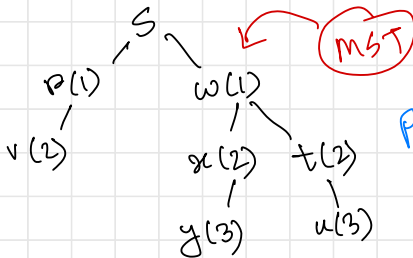
BFS → Shortest Path / MST (only for Unweighted)



Queue (FIFO)

s
~~x~~
~~w~~
~~t~~
~~u~~
~~y~~

a, b will remain white. They won't enter into queue.



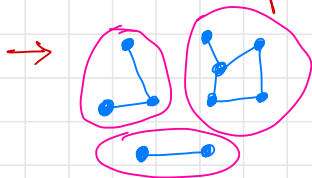
Path → s → y? (Shortest)
y ← x ← w ← s

Steps (BFS)

- Initially every vertex white
- After entering to queue it becomes grey
- Becomes black after entering every adjacent vertex

Runs until the stack is empty

Connected Component

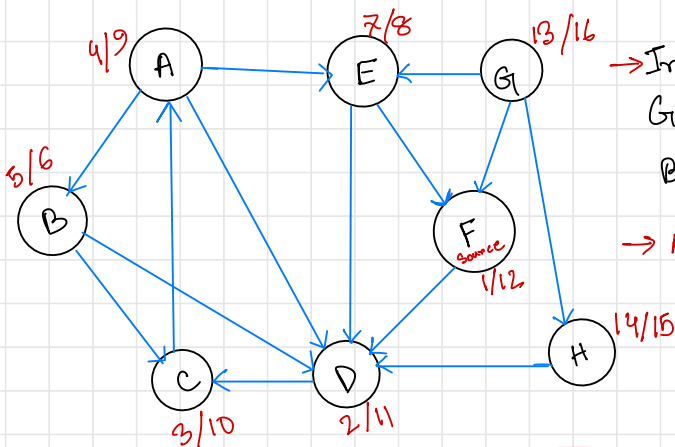


3 BFS Runs

$\therefore$ 3 Connected Component

Run BFS until every node is black. Count of BFS will be the count of connected component.

DFS



$\rightarrow$ Initially white

Grey when discovered

Black when finished

$\rightarrow$ After adjacents done it finishes

DV(F)

$\downarrow \uparrow$

DV(D)

$\downarrow \uparrow$

DV(C)

$\downarrow \uparrow$

DV(A)

$\downarrow \uparrow$

DV(B)

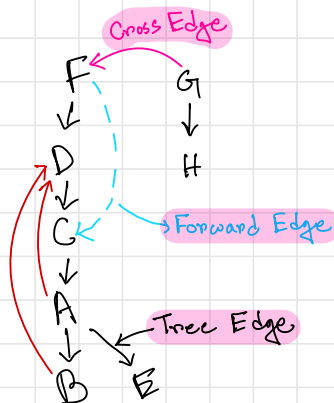
$\swarrow \nwarrow$

DV(G)

$\downarrow \uparrow$

DV(H)

$\downarrow \uparrow$



$\rightarrow$ Back Edge

Means there's a cycle

If a grey vertex visits another grey vertex then backedge happens

BFS(G, s)

```

1  for each vertex  $u \in V[G] - \{s\}$ 
2      do  $color[u] \leftarrow WHITE$ 
3       $d[u] \leftarrow \infty$ 
4       $\pi[u] \leftarrow NIL$ 
5   $color[s] \leftarrow GRAY$ 
6   $d[s] \leftarrow 0$ 
7   $\pi[s] \leftarrow NIL$ 
8   $Q \leftarrow \emptyset$ 
9  ENQUEUE( $Q, s$ )
10 while  $Q \neq \emptyset$ 
11     do  $u \leftarrow DEQUEUE(Q)$ 
12     for each  $v \in Adj[u]$ 
13         do if  $color[v] = WHITE$ 
14             then  $color[v] \leftarrow GRAY$ 
15                  $d[v] \leftarrow d[u] + 1$ 
16                  $\pi[v] \leftarrow u$ 
17                 ENQUEUE( $Q, v$ )
18      $color[u] \leftarrow BLACK$ 

```

$O(V)$

$O(1)$

$O(V)$

$O(E + V)$

$O(V)$

Total = $O(V + E)$

Line 12 { Undirected Graph $\rightarrow 2E + V$
Directed Graph $\rightarrow E + V$

DFS(G)

```

{
  for each vertex  $u \in G \rightarrow V$ 
  {
     $u \rightarrow color = WHITE$ ;
  }
  time = 0;
  for each vertex  $u \in G \rightarrow V$ 
  {
    if ( $u \rightarrow color == WHITE$ )
      DFS_Visit( $u$ );
  }
}

```

DFS_Visit(u)

```

{
   $u \rightarrow color = GREY$ ;
  time = time + 1;
   $u \rightarrow d = time$ ;
  for each  $v \in u \rightarrow Adj[]$ 
  {
    if ( $v \rightarrow color == WHITE$ )
      DFS_Visit( $v$ );
  }
   $u \rightarrow color = BLACK$ ;
  time = time + 1;
   $u \rightarrow f = time$ ;
}

```

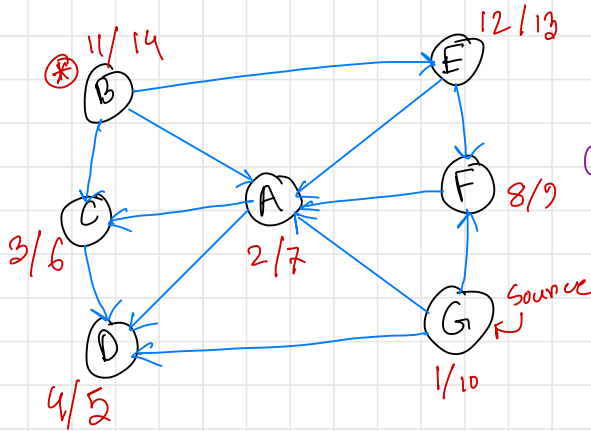
DAG $\rightarrow$ Directed Acyclic Graph (Directed Graph with no cycles)

Topological Sort

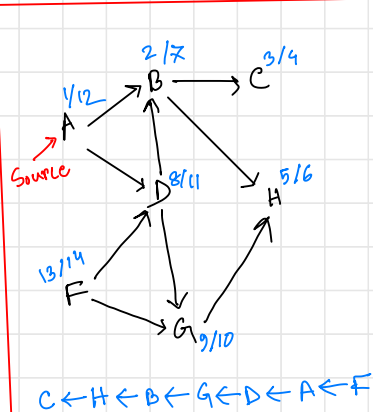
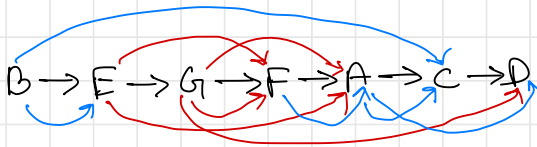
$\rightarrow$ Directed acyclic graph (DAG)

$\rightarrow$ Run DFS

$\rightarrow$ Put the finish on to top

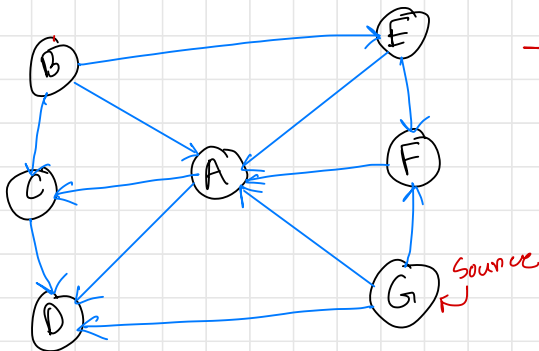


$G \leftarrow E \leftarrow B$
 $G \downarrow F \downarrow A \downarrow C \downarrow D$

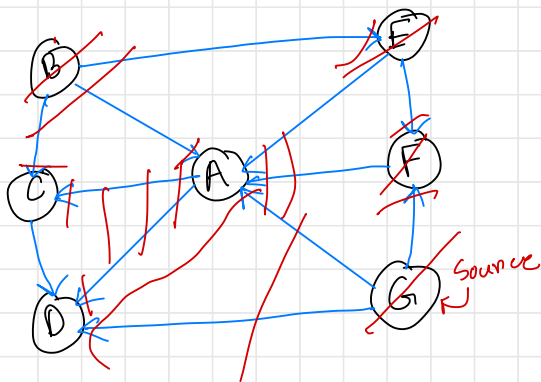


$C \leftarrow H \leftarrow B \leftarrow G \leftarrow D \leftarrow A \leftarrow F$

Topological Sort (Another Algorithm)



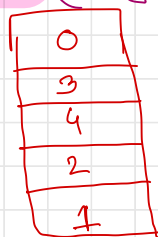
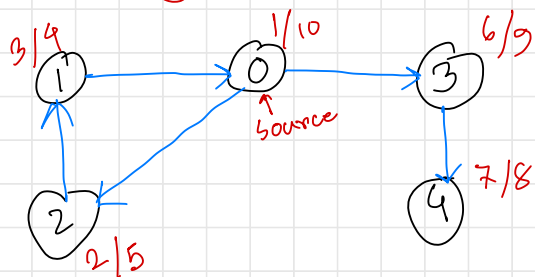
$\rightarrow$ Take the vertex who has no incoming edge.



Order:

G, B, E, F, A, C, D

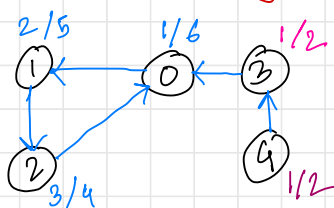
Strongly Connected Component (Every vertex is reachable from every other vertex)



Stack

→ Run DFS, ADD to stack which finishes first.

Now Reverse Every Direction of the graph



0	→ DFS {0, 1, 2}
3	→ DFS {3}
4	→ DFS {4}
2	x
1	x

Previous stack

3 Strongly Connected Component

→ Take out from stack one by one & Run DFS by considering it as source.

Disjoint Sets $\rightarrow$ Connected Component, MST

$\{ \{a, b, c\}, \{c, g, h\}, \{i\} \}$

$\rightarrow \text{MakeSet}(x) \rightarrow \{x\}$

$\rightarrow$ No common element

$\rightarrow \text{FindSet}(x) \rightarrow$ returns the representative of x

$\rightarrow \text{Union}(x, y) \rightarrow$ Merges x & y whole sets

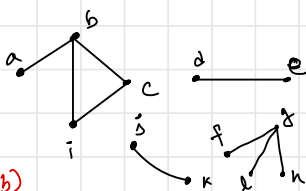
bool isSameSet(x, y)

return Find(x) == Find(y)

Connected Component

$\rightarrow$ Initially all vertex are separate sets

$\rightarrow \{a\} \{b\} \{c\} \{d\} \{e\}$
 $\{f\} \{g\} \{h\} \{i\} \{j\} \{k\} \{l\}$



$\rightarrow$ If edge(a, b) exist, Union(a, b)

$a-b: \{a, b\} \alpha$

$g-h: \{f, g, h\} \alpha$

$b-c: \{a, b, c\} \alpha$

$i-k: \{i, k\}$

$c-i: \{a, b, c, i\}$

$g-l: \{f, g, h, l\}$

$b-i: \alpha$

$d-e: \{d, e\}$

$f-g: \{f, g\} \alpha$

4 Connected Components

Vertex V
 Number of Union operation x
 Number of Connected Components k
 $V - x = k$

$$\Rightarrow 12 - x = 4$$

$$x = V - k$$

$$\Rightarrow x = 8$$

CONNECTED_COMPONENTS(G)

for each vertex v in $V[G]$ do

MAKE_SET(v) $\rightarrow O(V)$

for each edge (u, v) in $E[G]$ do

if FIND_SET(u) $\neq$ FIND_SET(v) then $\rightarrow 2E \rightarrow O(E)$

UNION(u, v) $\rightarrow O(V-k)$

SAME_COMPONENT(u, v)

if FIND_SET(u) == FIND_SET(v) then

return TRUE

else return FALSE

Disjoint Sets Implementation $\rightarrow$ Linked List

$\{a, b, d\}$: head $\rightarrow a \rightarrow b \rightarrow d$
 tail $\rightarrow$ back to representative

$\{c, e, g\}$: head $\rightarrow c \rightarrow g \rightarrow c$
 tail $\rightarrow$

$\{f\}$: head $\rightarrow f$
 tail $\rightarrow$

MakeSet(x)
 $h \rightarrow x$
 $t \rightarrow x$

Union($\{f, g\}$): head $\rightarrow f \rightarrow c \rightarrow g \rightarrow c$
 tail $\rightarrow$

$\rightarrow O(n^2)$

Weighted Union Heuristic $\rightarrow O(m + n \log n)$

$\rightarrow$ Append the shorter list to the longer list

Disjoint Sets Rooted Tree



a	b	e	f	g	k
b	b	e	b	g	g

$\rightarrow$ Root will be the representative

$\rightarrow$ Root's parent is itself

iv. Redraw the forest after Union(x, y).

Index	0	1	2	3	4	5	6	7	8	9	10	11
Parent	0	0	1	0	3	4	7	8	8	9	5	2

Table 1: Disjoint set (Rooted tree implementation)

Disjoint Forest



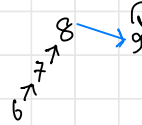
$$(i) \text{ Find}(5) = 5 \rightarrow 4 \rightarrow 3 \rightarrow 0 \rightarrow 0 = 0$$

$$\text{FindSet}(2) = 2 \rightarrow 1 \rightarrow 0 \rightarrow 0 = 0$$

$$(ii) \text{ Union}(6, 9) \rightarrow \text{Find}(6) = 8$$

$$\text{Find}(9) = 9$$

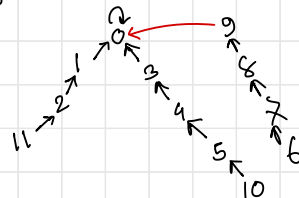
$$P[8] = 9$$



$$\text{Union}(9, 2) \rightarrow \text{Find}(9) = 9$$

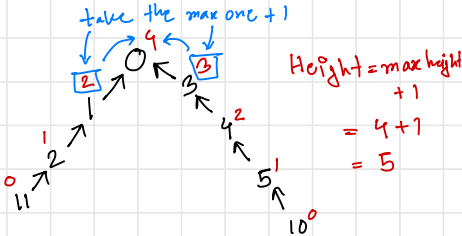
$$\text{Find}(2) = 0$$

$$P[9] = 0$$



Updated P	0	0	1	0	3	4	7	8	9	0	5	2
-----------	---	---	---	---	---	---	---	---	---	---	---	---

Path Compression:

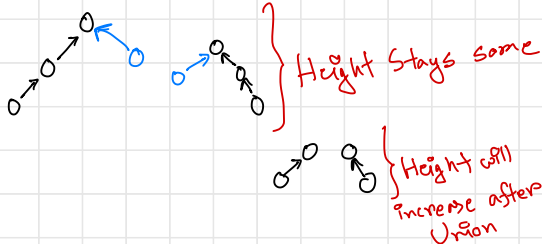


$\text{Find}(5) \rightarrow$



Union by Rank

Union(x, y)
Link(Find(x), Find(y))



Link(x, y)

If (rank[x] > rank[y])

P[y] = x

else {

P[x] = y

If (rank[x] == rank[y])

rank[y] ++;

}

N = 5 0, 1, 2, 3, 4 U(1, 2) U(4, 1)

⇒

	0	1	2	3	4
P	0	1	2	3	4
r	0	0	1	0	0

→ 1, 2, 3, 4, 0 (rank)

U(1, 2) → Link(1, 2)

U(4, 1) → Link(4, 2)

r[1] = 0

r[2] = 0

P[1] = 2

P[1] = P[2]

r[2] ++

r[4] = 0

r[2] = 1

P[4] = 2

P[4] < P[2]

2. (a) Table 1 shows the parent array of a Disjoint set (Rooted tree implementation). Perform the following operations sequentially using path compression and union-by-rank heuristic:

- Draw the disjoint set forest
- What will be returned by Find-Set(Y), and Find-Set(6)?
- Redraw the forest after Union(6, 9).
- Redraw the forest after Union(9, Y).

[2]
[1]
[1]
[1]

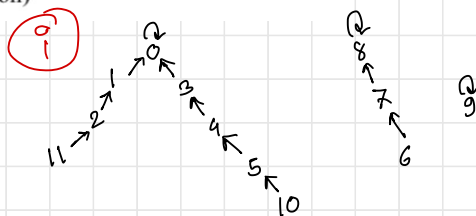
Index	0	1	2	3	4	5	6	7	8	9	10	11
Parent	0	0	1	0	3	4	7	8	8	9	X 5	Y 2

Table 1: Disjoint set (Rooted tree implementation)

Find-Set(2) → 2 → 1 → 0 → 0 = 0



Find-Set(6) → 6 → 7 → 8 → 8 = 8

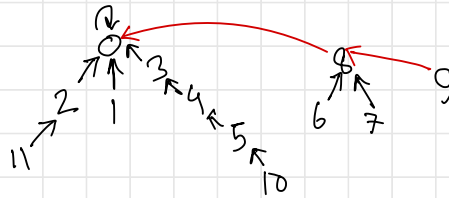


iii Union (6,9) $\rightarrow$ Find (6): $6 \rightarrow 8 \rightarrow 8 = 8$

Find (9) = 9

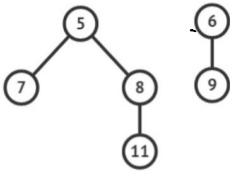
iv Union (9,2) $\rightarrow$ Find (9): $9 \rightarrow 8 \rightarrow 8 = 8$

Find (2): $2 \rightarrow 0 \rightarrow 0 = 0$

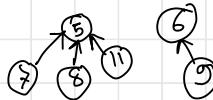
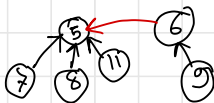


high rank
low rank

- 4 Draw the resultant forest after calling UNION(6, 11) and after that, FIND-SET(9) [3]
on the disjoint-sets of the following figure. You must use the union-by-rank and the path-compression heuristics.



Union (6,11) $\rightarrow$ Find (6) = 6
Find (11) = $11 \rightarrow 8 \rightarrow 5 = 5$



Find (9) $\rightarrow 9 \rightarrow 6 \rightarrow 5 = 5$

(b) Perform the following operations for a disjoint set *sequentially*. Assume the elements of the disjoint set are 0 to 7. Apply *union-by-rank* and *path-compression* where applicable.

(i) Makeset(0)

(ii) Makeset(1)

(iii) Makeset(2)

(iv) Makeset(3)

(v) Makeset(4)

(vi) Makeset(5)

(vii) Makeset(6)

(viii) Makeset(7)

(ix) Union(**x**, **y**)

(x) Union(**y**, **y** + 1)

(xi) Draw the disjoint set forest.

(xii) Find_set(**x**)

$$x = 5 \\ y = 2$$

$\Rightarrow \{0\} \{1\} \{2\} \{3\} \{4\} \{5\} \{6\} \{7\}$

Union(5, 2)

Find(5) = 5
Find(2) = 2

Union(2, 3)

Find(2) = 2
Find(3) = 2

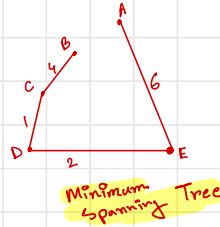
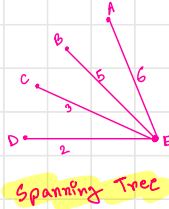
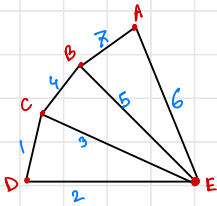
Find(5) : $5 \rightarrow 2 \rightarrow 2 = 2$

MST (Minimum Spanning Tree)

Graph must be:

Undirected
weighted
Connected

[Edge $v-1$]

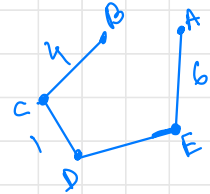


Kruskal's Algorithm

Steps:

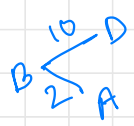
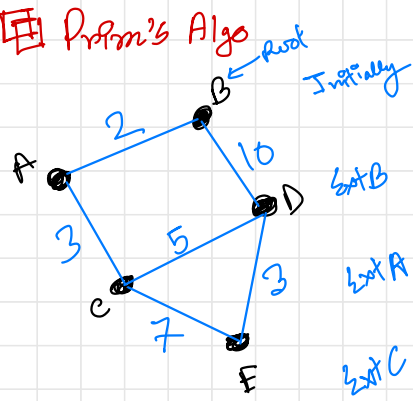
- Make every vertex different sets
- Sort every edges ascending by weight
- Take the edge one by one, do union if they don't belong to the same set

$C \xrightarrow{1} D$ ✓ {C,D}
 $D \xrightarrow{2} E$ ✓ {C,D,E}
 $C \xrightarrow{3} E$ X
 $C \xrightarrow{4} B$ ✓ {C,D,E,B}
 $B \xrightarrow{5} E$ X
 $A \xrightarrow{6} E$ ✓ {C,D,E,B,A}
 $B \xrightarrow{7} A$ X



Prim's algorithm is very similar to Kruskal's: whereas Kruskal's "grows" a forest of trees, Prim's algorithm grows a single tree until it becomes the minimum spanning tree. Both algorithms use the greedy approach - they add the cheapest edge that will not cause a cycle. But rather than choosing the cheapest edge that will connect any pair of trees together, Prim's algorithm only adds edges that join nodes to the existing tree. (In this respect, Prim's algorithm is very similar to Dijkstra's algorithm for finding shortest paths.)

Prim's Algo



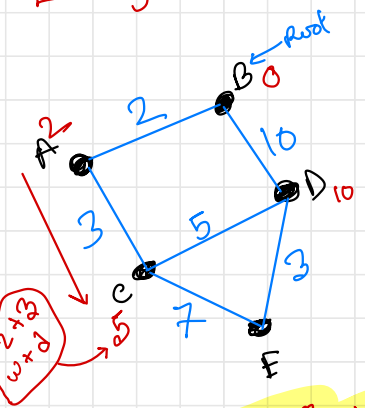
D: D in Queue? ✓
 $10 < \infty$
 $k[D] = 10$
 $P[D] = B$

Queue
 A ∞
 B ϕ
 C ∞
 D ∞
 E ∞
 F ∞

B
 2 /
 A B
 3 /
 C
 5 /
 D
 3 /
 E

A	B	C	D	E	
∞ 2	ϕ	∞	∞ 10	∞	
2, B			10, B		
		3, A			
			5, C	7, C	
				3, D	

Dijkstra Algo



Initially

Ext B

Ext A

Ext C

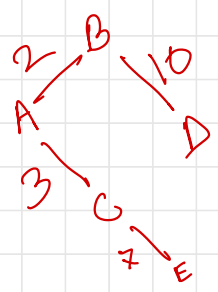
Ext D

Ext E

Minimum weight + distance

A	B	C	D	E	
∞ ϕ	0 ϕ	∞ ϕ	∞ ϕ	∞ ϕ	
(2) B			(10) B		
		Weight (5) A			
				12, C	

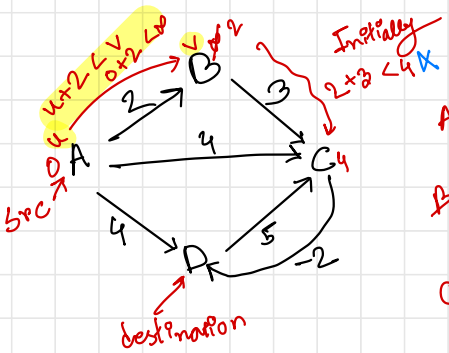
- Queue
- ~~A~~
 - ~~B~~
 - ~~C~~
 - D
 - E



All Possible Shortest Path from B

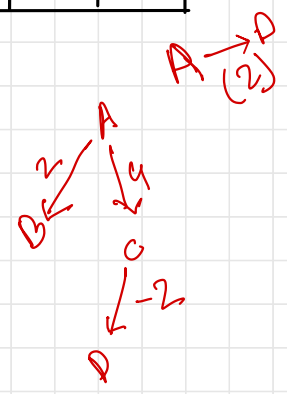
- B = A
- B - A - C (5)
- B - A - C - E (12)

Bellman Ford



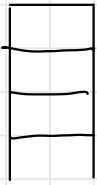
	A	B	C	D	
A	0	2, A	4, A	4, A	
B	0	2, A	4, A	4, A	
C	0	2, A	4, A	2, C	
D	0	2, A	4, A	2, C	

	A	B	C	D		
A	0	2, A	4, A	2, C		
B	0	"	"	"		
C	0	"	"	"		
D	0	"	"	"		



Hash Table

⊕ Direct Address Table



★ waste of space

★ Infeasibility (unknown Range)

Solution: Hashing

⊕ $h(x) = (x+5) \% 6 \leftarrow$ Hash Function

0	1	2	3	4	5
		15		11	6

5

input $\rightarrow 11, 15, 6, 5, 1065$

$$h(11) = (11+5) \% 6 = 16 \% 6 = 4$$

$$h(15) = (15+5) \% 6 = 20 \% 6 = 2$$

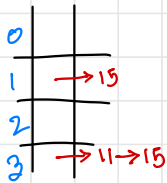
$$h(6) = (6+5) \% 6 = 11 \% 6 = 5$$

$$h(5) = (5+5) \% 6 = 10 \% 6 = 4$$

Collision

Collision Solve:

⊕ Chaining (Array of Linked Lists)



⊕ No limit to storing numbers

⊖ Waste of space

Insertion $\rightarrow O(1)$

Search/Delete $\rightarrow$ load factor, $\alpha = \frac{n}{m}$



Average $\rightarrow O(1 + \alpha)$ (Not Found)
 $O(1 + \frac{\alpha}{2})$ (Found)

Worst $\rightarrow O(n)$

⊕ Open Addressing

0	1	2	3	4	5
5		15		11	6

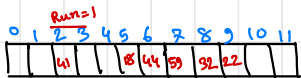
⊖ Cannot accommodate many numbers (Limited Space)

⊕ Utilize of space

⊖ Has a problem with deletion

Probing [Find Blank Box]

3 Types of Open Addressing:



① Linear Probing $\rightarrow$ Primary Clustering $\rightarrow$ Run = 5 $\rightarrow h(k, i) = (h'(k) + i) \% m$

② Quadratic Problem $\rightarrow$ Secondary Clustering  $\rightarrow h(k, i) = (h'(k) + 2i^2) \% m$

(iii) Double Hashing $\rightarrow$ No Cluster

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \% m$$

Linear Probing

$$h'(k) = k \text{ mod } m$$

hash table size

$$h(k,i) = (h'(k) + i) \% m$$

count of

0	1	2	3	4	5	6	7	8	9	10	11	12

run=1
primary run=5 → clustering

Inversion

Insertion

181. $h'(18) = 18 \cdot 13 = 5$

190. $h(k, i) = (5+0) \cdot 13 = 5$

41. $h'(41) = 41 \cdot 13 = 2$

190. $h(k, i) = (2+0) \cdot 13 = 2$

22. $h'(22) = 22 \cdot 13 = 9$

44. $h(k, i) = (9+0 \cdot 13) = 9$

190. $h(k, i) = (9+0 \cdot 13) = 9$

190. $h(k, i) = (5+0) \cdot 13 = 5$

190. $h(k, i) = (5+1) \cdot 13 = 6$

501. $h'(59) = 59 \cdot 13 = 2$

190. $h(k, i) = 7$

32. $h'(32) = 6$

190. $h(k, i) = 6$

190. $h(k, i) = 7$

190. $h(k, i) = 8$

Searching

Search 44 $h(44) = 5$

$i = 0$ $h(44, 0) = 5$

$i = 1$ $h(44, 1) = 6$ (Found)

Search 70 : $h(70) = 5$

$i = 0$ $h(70, 0) = 5$

1 6

2 7

3 8

4 9

Until finds any
Empty box

(Not NIL / Empty)

Delete

Delete 23/24 Box 6 Deleted 23/24 (not 20)

(Increasing AS)

Deleted $\rightarrow$ Insert empty

Search full

Quadratic Probing

$h(k) = k \bmod 13$

$h(k, i) = (h(k) + 2 \cdot i^2) \bmod 13$

$$h(k, i) = (h_1(k) + 9h_2(k)) \bmod 7$$

$$h_1(k) = k \% 7 \quad (0-6)$$

$$h_1(k) = k \cdot 7$$

$$h_2(k) = 5 - (k \% 5)$$

Keys: 76, 40, 47, 55, 10, 93

$\Rightarrow i=0 \quad u_1(76) = 76 : 7 = 6$

$$h(k, i) = (6 + 0 \times h_2) \times 7 = 6$$

$$i=0 \quad w_1(40) = 40 \cdot 1/7 = 5$$

$$h(k, p) = 5 \cdot 7 = 5$$

$$i=0 \quad h_1(47) = 47 \div 7 = 5 \quad h(1,0) = (5+3) \div 7 = 1$$

$$p_2 \quad h_2(47) = 5 - (47 \times 5) = 5 - 2 = 3$$

0	1	2	3	4	5	6
	47	93	10	55	40	76

$$i=0 \quad h(55) = 55 \div 7 = 6$$

$$T=1 \quad h_2(55) = 5 - (55 \div 5) = 5$$

$$h(k,i) = 11 \cdot 7 = 4$$

$$\hat{\tau}=0 \quad h_1(10) = 10 \cdot 7 = 3$$

$$i=0 \quad w_1(93) = 93/7 = 2$$

$q=0$
 Delete(47): $h_1(47) = 47 \div 7$
 $= 5$ ✗

$$i=1 \quad \log_2(47) = 3$$

$$h(k, i) = 1 \text{ [Found at 1]}$$

Rabin Karp Algorithm

RABIN-KARP-MATCHER(T, P, d, q)

```

1   $n \leftarrow \text{length}[T]$ 
2   $m \leftarrow \text{length}[P]$ 
3   $h \leftarrow d^{m-1} \bmod q$ 
4   $p \leftarrow 0$ 
5   $t_0 \leftarrow 0$ 
6  for  $i \leftarrow 1$  to  $m$ 
7      do  $p \leftarrow (dp + P[i]) \bmod q$ 
8       $t_0 \leftarrow (dt_0 + T[i]) \bmod q$ 
9  for  $s \leftarrow 0$  to  $n - m$ 
10     do if  $p = t_s$ 
11         then if  $P[1..m] = T[s+1..s+m]$ 
12             then "Pattern occurs with shift"  $s$ 
13         if  $s < n - m$ 
14             then  $t_{s+1} \leftarrow (d(t_s - T[s+1]h) + T[s+m+1]) \bmod q$ 
    
```

n = Text Length

m = Pattern Length

$h = d^{m-1} \% q$, $d = 10 / \text{highest} + 1$

Let q as a prime number

```

for  $i \leftarrow 1$  to  $m$ 
do  $p \leftarrow (dp + P[i]) \bmod q$ 
    $t_0 \leftarrow (dt_0 + T[i]) \bmod q$ 
    
```

→ Preprocessing the first hash value

$t_{s+1} = (d(t_s - T[s+1]h) + T[s+m+1]) \bmod q$,

→ We can get all the values in $O(1)$ time using this formula

☑ $T = \text{"001014 AABABE"} \quad n = 6 \quad \text{Let } q = 5$

$P = \text{"ABC"} \quad m = 3 \quad d = 10, h = d^{m-1} \% q = 10^2 \% 5 = 0$

$S = n - m + 1 = 6 - 3 + 1 = 4$

AABABE
 $s = 4$

$p = 0, t_0 = 0$

for $i \leftarrow 1$ to m
do $p \leftarrow (dp + P[i]) \bmod q$
 $t_0 \leftarrow (dt_0 + T[i]) \bmod q$

$i = 1 \rightarrow p = (dp + P[1]) \% q = (10 \times 0 + 0) \% 5 = 0$

$t_0 = (dt_0 + T[1]) \% q = (10 \times 0 + 0) \% 5 = 0$

$i = 2 \rightarrow p = (10 \times 0 + 1) \% 5 = 1$

$t_0 = (10 \times 0 + 0) \% 5 = 0$

$i = 3 \rightarrow p = (10 \times 1 + 2) \% 5 = 2$

$t_0 = (10 \times 0 + 1) \% 5 = 1$

$S = 0: p = t_0? \quad 2 \neq 1$

No match!

91 means
 $012 \% 5 = 2$
 $001 \% 5 = 1$

$$t_{s+1} = (d(t_s - T[s+1]h) + T[s+m+1]) \bmod q$$

t_0
AAGABE $\rightarrow$ Index 1, 2, 3, 4, ...
A - AB + A t_1
A - AB + A t_2
A - AB + A t_3

$$t_1 = d \times (t_0 - T[1] \times h) + T[0+3+1] \% 5$$

$$= (10 \times (1 - 0 \times 0) + 0) \% 5$$

$$= 0$$

$s=1$: $P \neq t_1 \Rightarrow 2 \neq 0$ No match

$$t_2 = d \times (t_1 - T[2] \times h) + T[5] \% 5$$

$$= (10 \times (0 - 0 \times 0) + 1) \% 5$$

$$= 1$$

$s=2$: $P \neq t_2 \Rightarrow 2 \neq 1$ No match

$$t_3 = d \times (t_2 - T[3] \times h) + T[6] \% 5$$

$$= (10 \times (1 - 1 \times 0) + 4) \% 5$$

$$= 14 \% 5 = 4$$

$s=3$: $P \neq t_3$ No match

(d) Given a test string: **fedabdea** and a pattern: **dac**, use Rabin-Karp's algorithm to find whether the pattern exists in the string or not. You must show the Hash values and calculations for all the substrings. The values of each character are given below.

[2]

char	a	b	c	d	e	f
value	1	2	3	4	5	6

$$7 \overline{) 413} \begin{array}{r} 59 \\ 35 \\ \hline 68 \\ 63 \\ \hline 5 \end{array}$$

fedabdea $n=8$
63412451

dac $m=3$
413

$$d=10 \quad h=d^{m-1} = 10^2 \% 7$$

$$= 100 \% 7$$

$$= 2$$

$$7 \overline{) 1100114} \begin{array}{r} 1571 \\ 70 \\ \hline 400 \\ 350 \\ \hline 501 \\ 490 \\ \hline 114 \\ 70 \\ \hline 44 \\ 35 \\ \hline 9 \end{array}$$

$$P=0 \quad t_0=0$$

$$P=1 \quad P = dP + P[1] \% 7$$

$$= (10 \times 0 + 4) \% 7$$

$$= 4$$

$$t_0 = dt_0 + T[1] \% 7$$

$$= 10 \times 0 + 6 \% 7 = 6$$

$$P=2 \quad P = dP + P[2] \% 7$$

$$= 10 \times 4 + 1 \% 7$$

$$= 41 \% 7 = 6$$

$$t_0 = dt_0 + T[2] \% 7$$

$$= 10 \times 6 + 3 \% 7 = 0$$

$$P=3 \quad P = dP + P[3] \% 7$$

$$= 10 \times 6 + 2 \% 7$$

$$= 62 \% 7$$

$$= 0$$

$$t_0 = 10 \times 0 + T[3] \% 7$$

$$= 4 \% 7 = 4$$

$$S=0 \Rightarrow P=0 \quad t_0=4$$

$$\therefore P \neq t_0 \text{ (No match)}$$

$$t_1 = d \times (t_0 - T[1] \times h) + T[4] \% 7$$

$$= (10 \times (4 - 6 \times 2) + 1) \% 7$$

$$= (10 \times (-8) + 1) \% 7$$

$$= -79 \% 7 \quad \left| \begin{array}{l} 12 \times 7 = 84 \\ 84 - 79 = 5 \end{array} \right.$$

$$= 5$$

$$S=1 \Rightarrow P=0 \quad t_1=5 \quad P \neq t_1 \text{ No match}$$

$$t_2 = d \times (t_1 - T[2] \times h) + T[5] \% 7$$

$$= 10 \times (5 - 3 \times 2) + 2 \% 7$$

$$= 10 \times (-1) + 2 \% 7$$

$$= -8 \% 7 \quad \left| \begin{array}{l} 14 - 8 = 6 \end{array} \right.$$

$$= 6$$

$$S=2 \quad P \neq t_2 \text{ No Match}$$

$$t_3 = 10 \times (6 - 4 \times 2) + 4 \% 7$$

$$= -16 \% 7 \quad \left| \begin{array}{l} 21 - 16 = 5 \end{array} \right.$$

$$= 5$$

$$S=3 \quad P \neq t_3 \text{ No match}$$

$$t_4 = 10 \times (5 - 1 \times 2) + 5 \% 7$$

$$= 35 \% 7$$

$$= 0$$

$$S=4 \quad P = t_4 \quad \text{SpurPow} > HPt$$

63412451
fc ab dea

$$S = n - m + 1$$

$$= 8 - 3 + 1$$

$$= 6$$

$$-81 \% 7 ?$$

→ Find immediate lowest than -81 which is divisible by 7.

$$(-84)$$

→ Make it positive and subtract it. That's the MOD
 $84 - 81 = 3$

$$\# -3 \% 8$$

$$(-8)$$

$$\rightarrow 8 - 3$$

$$= 5$$

$$t_5 = 10 \times (0 - 2 \times 2) + 1 \% 7$$

$$= -39 \% 7 \quad \left| \begin{array}{l} 42 - 39 = 3 \end{array} \right.$$

$$= 3$$

$$P \neq t_5 \quad \text{No Match}$$



United International University (UIU)

Dept. of Computer Science & Engineering (CSE)

Final Exam Total Marks: 40 Summer 2022

Course Code: CSE 2217

Course Title: Data Structure and Algorithms II

Time: 2 hours

Any examinee found adopting unfair means will be expelled from the trimester / program as per UIU disciplinary rules.

There are SEVEN questions. Answer all of them. Show full simulation or solved graph figures wherever necessary. Figures in the right-hand margin indicate full marks.

✓ 1.	(a) Fig. 1(a) is a graph where nodes A, B, C and D contain values 1, 2, 4 and 0 respectively. The BFS and DFS trees of the graph are in Fig. 1(b) and Fig. 1(c) respectively. So the paths of the BFS tree form the numbers 120 and 14 while the paths of the DFS tree form the number 1204. Given this formulation, what will be the graph if the BFS tree forms the numbers 1246, 1247 and 13 and the DFS tree forms the numbers 12467 and 13? You can assume that each node will have a unique value within 0 to 9 and the graph will contain no more than 10 nodes. Fig. 1(a) Fig. 1(b) Fig. 1(c)	[2]
✓ 2.	(b) Can you find the shortest path from A to C in the following graph (Fig. 2) using BFS? Explain your answer briefly. Fig. 2. A directed graph	[1]
✓ 3.	(c) “Breadth First Search” can create a minimum spanning tree for unweighted graphs. Explain why this statement is true. MST Any spanning tree is a MST for unweighted graph	[2]

$$\begin{array}{c} A-B-C \\ A-C \end{array} \quad \left. \begin{array}{l} \text{Same} \\ 1 \end{array} \right\} \text{Any spanning tree is a MST for unweighted graph}$$

$\text{Union}(x, y)$
 $\text{Link}(\text{find}(x), \text{find}(y))$
 $\text{FindSets}(x) \begin{cases} \text{if } x \neq P[x] \\ P[x] = \text{FindSet}(P[x]) \end{cases}$
 $\text{return } P[x]$

- 2 (a) What is **union-by-rank** in the Disjoint-Set Union data structure? Why do we need this heuristic? Justify your answer. [2]
- (b) Write the pseudocode of **Union Sets** and **Find Sets** operation of Disjoint-Set Union data structure, assuming the *path-compression* and *union-by-rank* heuristics. [3]
- (c) Show the status of Fig. 3 after each of the following operations: **find_set(3)**, **find_set(6)**, **find_set(5)**, **find_set(3)**. Note that the operations are executed sequentially, and you must use the *path-compression heuristic*. [2]

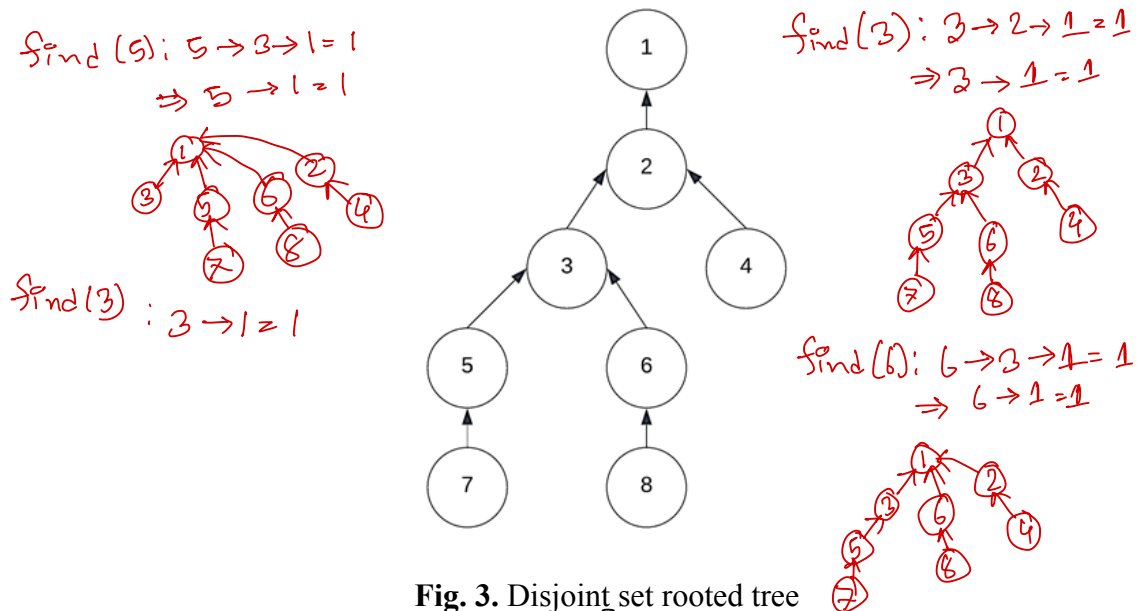


Fig. 3. Disjoint set rooted tree

- 3 (a) What will be the runtime of Kruskal's algorithm assuming edges are sorted using bubble sort? Note that bubble sort takes $O(n^2)$ time to sort n elements. [1]
- (b) Apply *Prim's algorithm* to find the MST from the graph in Fig. 4, assuming vertex **D** as the root. Show details and clearly draw the MST you have found. [4]

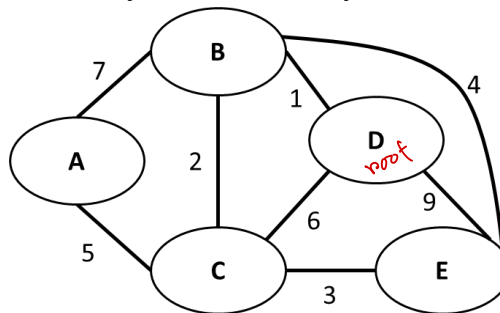


Fig. 4. An undirected graph

	A	B	C	D	E
Initially	∞	∞	∞	0	∞
Ext D		1, D	6, D		9, D
Ext B	7, B		2, B		4, B
Ext C	5, C				3, C
Ext E					
Ext A					

Handwritten red notes: A path 1-0-2-D is shown, and a list of rejected edges: 5, 7, 9, 6.

- (c) "Kruskal's algorithm grows a forest of trees and eventually forms the MST." - Explain this statement with an example. [2]
- 4 (a) A graph contains the vertices $\{A, B, C, D, E, F, G\}$ and the shortest path from A to B is $A \rightarrow E \rightarrow C \rightarrow D \rightarrow F \rightarrow G \rightarrow B$. Is it possible to find the shortest path from E to F from given data? Justify your answer. [2]

(b) Find the **Shortest path tree** for the given graph in **Fig. 5**, where the source is vertex **z**. Show the details of your calculation. **Justify** how the shortest path tree satisfies *optimal substructure property*.

[3+2]

Iteration - 1

	a	b	z	x	y
Init.	∞	∞	0	∞	∞
z	6, z			7, z	
a					
x		4, x			
y					
b	2, b				

Iteration - 2

	a	b	z	x	y
z	2, b	4, x	0	7, z	∞
a					
b					
z					
x					
y					

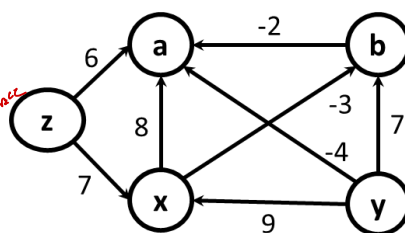


Fig. 5: A directed graph $G(V,E)$

5 (a) What are the benefits of a good Hash Function? $\rightarrow$ fast, collision, easy to compute [1]
 (b) Which data structure is best suited for implementing Chaining in a HashTable: Binary Search Tree or Linked List? Explain with time complexity comparisons between the both for operations such as Insertion, Search and Deletion. [3]

6 (a) Explain the working of the Rolling Hash Function in the Rabin-Karp algorithm using a suitable numerical example. [3]

(d) Given a test string: **fcadbdea** and a pattern: **dac**, use Rabin-Karp's algorithm to find whether the pattern exists in the string or not. You must show the Hash values and calculations for all the substrings. The values of each character are given below. [2]

char	a	b	c	d	e	f
value	1	2	3	4	5	6

7. (a) Write the definitions of the following terms:

- NP-Complete
- P
- NP-hard

(b) What does the following diagram (**Fig. 6**) represent? Explain.

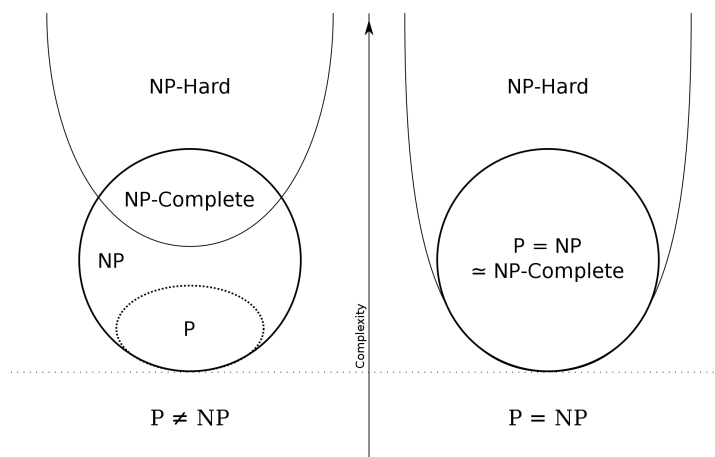


Fig. 6: Question 7(b)



United International University (UIU)

Dept. of Computer Science & Engineering (CSE)

Final Exam Total Marks: 40 Spring 2022

Course Code: CSE 2217

Course Title: Data Structure and Algorithms II

Time: 2 hours

Any examinee found adopting unfair means will be expelled from the trimester / program as per UIU disciplinary rules.

There are **SEVEN** questions. **Answer all of them.** Show full simulation and/or solved figures wherever necessary. Figures in the right-hand margin indicate full marks.

	A	B	C	D	E	F	G	H	I	J
A	0	∞	∞	∞	∞	∞	∞	∞	∞	∞
B	15, A	0	∞	∞	∞	∞	∞	∞	∞	∞
C	10, B	7, A	0	∞	∞	∞	∞	∞	∞	∞
D	4, C	10, C	3, C	0	∞	∞	∞	∞	∞	∞
E					0	∞	∞	∞	∞	∞
F						0	∞	∞	∞	∞
G							0	∞	∞	∞
H								0	∞	∞
I									0	∞
J										0

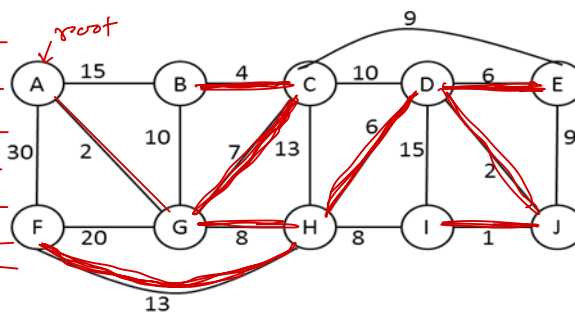


Figure 1: An undirected graph $G(V,E)$

1 (a) Find the MST for the given graph in Figure 1 using **Prim's algorithm**. Show the details of your calculation. Start from **node A**. [3]

(b) Given a weighted graph where multiple edges have the same weight, you are asked to apply **Kruskal's algorithm** to find MST. You have chosen the merge sort algorithm to sort the edges in ascending order. "Depending on the sequence of the sorted edges, you may have different MSTs". Is the statement **True** or **False**? **Justify** your answer. [1]

(c) Suppose you are asked to utilize a disjoint set data structure to implement **Kruskal's algorithm** to find MST from a given graph. **Explain, with an example**, how disjoint set data structure would help you to perform the following operations: [2]

- joining two trees in the forest $\{a,b\} \cup \{c\} \rightarrow \{a,b,c\}$
- detecting a cycle $\{a,b\} \cup \{c\} \rightarrow \{a,b,c\}$ (if a,b are already in the same set, adding c creates a cycle)

2 (a) Find the **Shortest path tree** for the given graph in Figure 2, where the source is vertex **y**. Show the details of your calculation. [3]

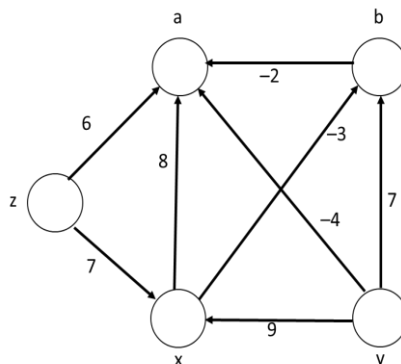
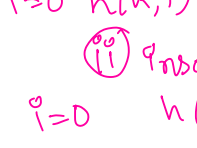
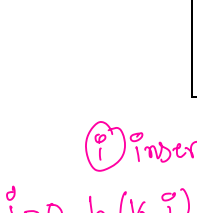
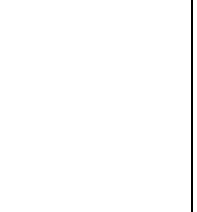
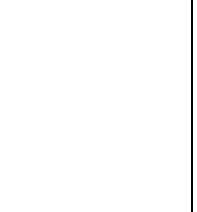
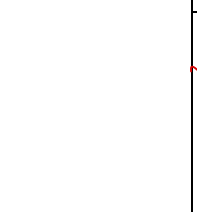
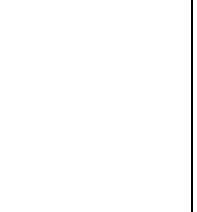
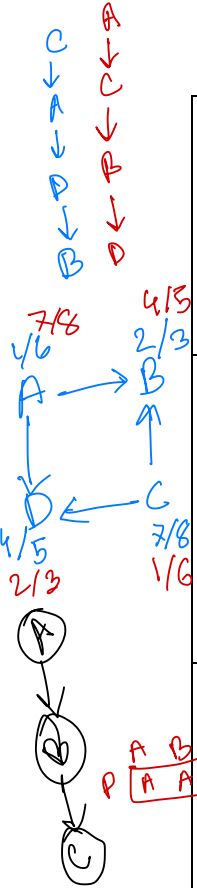


Figure 2: A directed graph $G(V,E)$

	(b) A graph contains the vertices {A, B, C, D, E, F, G} and the shortest path from A to B is $A \rightarrow E \rightarrow C \rightarrow D \rightarrow F \rightarrow G \rightarrow B$. Is it possible to find the shortest path from E to F from given data? Justify your answer. [1] (c) What do you understand by <i>edge relaxation</i>? Let $d(v)$ be the shortest path estimate from node s to node v and $\delta(s, v)$ be the shortest path value from node s to node v. When does it happen that $d[v] = \delta(s, v)$? [2]	
3	(a) Show the values for <i>finish time</i> (f) and <i>parent</i> (π) that result from running Depth-First Search (DFS) on the undirected graph in Figure 1, using vertex A as the source vertex. [3] (b) "Every connected directed acyclic graph (DAG) has exactly one topological ordering" - true or false? Design a graph G with exactly 4 vertices that justifies your reasoning. [2]	
4	(a) Why do we use the heuristics: <i>union-by-rank</i> and <i>path-compression</i> in Disjoint-Set data structure? [3] (b) Draw the resultant forest after calling UNION(5, 6) and after that draw the resultant forest again after calling FIND-SET(1) on the disjoint-sets of the following figure. You must use the <i>union-by-rank</i> and the <i>path-compression</i> heuristics. [3]	
5	(a) What is <i>Secondary Clustering</i>? When might a hash table face this suffering? [1] (b) Consider an open-addressing hash table as shown below. The table already contains some data items and other empty slots. Assume that collisions are handled by Quadratic probing using the hash function $h(k, i) = (h'(k) + i^2) \bmod 13$, where $h'(k) = (k + 7) \bmod 13$. By showing detailed calculations, redraw the table after (i) Insert 47 [1] (ii) insert 64; [1] (iii) delete 12 (replace with NIL); [1] (iv) search 38. [1] Although 38 had been present at the hash table, your search would fail here. Explain how one can modify the operations such that this will be prevented. [1]	



(i) insert (47) $\rightarrow h'(k) = (47 + 7) \% 13 = 2$
 $i = 0 \quad h(k, i) = (2 + 0) \% 13 = 2$
 (ii) insert (64) $\rightarrow h'(k) = (64 + 7) \% 13 = 6$
 $i = 0 \quad h(k, i) = (6 + 0) \% 13 = 6$
 (iii) delete (12) $\rightarrow h'(k) = (12 + 7) \% 13 = 6$
 $i = 0 \quad h(k, i) = (6 + 0) \% 13 = 6$
 (iv) search (38) $\rightarrow h'(k) = (38 + 7) \% 13 = 5$
 $i = 0 \quad h(k, i) = (5 + 0) \% 13 = 5$
 $i = 1 \quad h(k, i) = (5 + 1) \% 13 = 6$
 $i = 2 \quad h(k, i) = (5 + 4) \% 13 = 10$
 $i = 3 \quad h(k, i) = (5 + 9) \% 13 = 12$
 $i = 4 \quad h(k, i) = (5 + 16) \% 13 = 7$
 $i = 5 \quad h(k, i) = (5 + 25) \% 13 = 11$
 $i = 6 \quad h(k, i) = (5 + 36) \% 13 = 8$
 $i = 7 \quad h(k, i) = (5 + 49) \% 13 = 1$
 $i = 8 \quad h(k, i) = (5 + 64) \% 13 = 10$
 $i = 9 \quad h(k, i) = (5 + 81) \% 13 = 4$
 $i = 10 \quad h(k, i) = (5 + 100) \% 13 = 9$
 $i = 11 \quad h(k, i) = (5 + 121) \% 13 = 3$
 $i = 12 \quad h(k, i) = (5 + 144) \% 13 = 12$

6	Consider the mentioned notations Text, $t = \text{"BATMANCATWOMAN"}$ Pattern, $p = \text{"AT"}$ Modulo, $q = 13$ Hash of a string XY, $h(XY) = (X + Y) \bmod q$; where X and Y are the alphabetical sequence of keys [i.e.: $A=1, B=2, C=3, \dots Y=25, Z=26$ etc] Now answer the following questions using Rabin-Karp String matching algorithm: (a) Show the indices of <i>valid hit</i> and <i>spurious hit</i> (if any) using the aforementioned hashing function. [2+1] (b) What is the time complexity of this algorithm? Explain how it improves the naïve string-matching algorithm? [2]	
7	(a) When does a problem belong to the complexity class NP? How does it differ from P class problems? [2] (b) "Being $P \neq NP$ drives the world economy and gives the modern Password protection system a profound advantage." Do you agree with this? Briefly explain your opinion. [4]	

6) $t = \overset{2}{B} \overset{1}{A} \overset{20}{T} \overset{13}{M} \overset{1}{A} \overset{14}{N} \overset{3}{C} \overset{1}{A} \overset{20}{T} \overset{23}{W} \overset{15}{O} \overset{13}{M} \overset{1}{A} \overset{14}{N}$
 $p = \overset{1}{A} \overset{20}{T} \Rightarrow 120 \quad S = 14 - 2 + 1 = 13 \quad n = 14 \quad m = 2, q = 13$
 $d = 27 \quad h = d^{m-1} = 27^2 \% 13 = 1$
 $P(AT) = (1 + 20) \% 13 = 21 \% 13 = 8$
 $T(BA) = (2 + 1) \% 13 = 3 \% 13 = 3$
 $T(AT) = (1 + 20) \% 13 = 8 \text{ (match)}$
 $T(TM) = (20 + 13) \% 13 = 7$
 $T(MA) = (13 + 1) \% 13 = 1$
 $T(AN) = (1 + 14) \% 13 = 2$
 $T(NC) = (14 + 3) \% 13 = 4$
 $T(CA) = (3 + 1) \% 13 = 4$
 $T(AT) = (1 + 20) \% 13 = 8 \text{ (match)}$
 $T(TW) = (20 + 23) \% 13 = 4$
 $T(WO) = (23 + 15) \% 13 = 12$
 $T(OM) = (15 + 13) \% 13 = 2$
 $T(MA) = (13 + 1) \% 13 = 1$
 $T(AN) = (1 + 14) \% 13 = 2$

You have to answer here. No extra page will be provided.Use the value of x in Question 1, where $X = \text{last_digit_of_your_student_id} = \underline{3}$

1. Consider an open-addressing hash table as shown below (Table 1). The table already contains three data items. Assume that collisions are handled by the following hash function.

$$h(k, i) = (h'(k) + i h_2(k)) \bmod 13, \rightarrow \text{Double hashing}$$

$$\text{where } h'(k) = (2k + 7) \bmod 13 \quad \text{and} \quad h_2(k) = (k + 5) \bmod 13.$$

By showing calculations, redraw Table 2 and show following operations

[2+2+2+2]

- i. Insert 70
- ii. Insert 83
- iii. Insert $X \rightarrow 3$
- iv. Search 99

0	1	2	3	4	5	6	7	8	9	10	11	12
3	70			44	12				51		83	

Table 1: Open Addressing Table

i. Insert (70):

$$h'(70) = 2 \times 70 + 7 \bmod 13$$

$$= 147 \bmod 13$$

$$= 4$$

$$i=0 \quad h(k, i) = (4 + 0) \bmod 13 = 4$$

$$i=1 \quad h_2(70) = 70 + 5 \bmod 13 = 10$$

$$h(k, i) = (4 + 10) \bmod 13 = 14 \bmod 13 = 1$$

ii. Insert (83):

$$h'(83) = (2 \times 83 + 7) \bmod 13 = 173 \bmod 13 = 4$$

$$h_2(83) = 83 + 5 \bmod 13 = 88 \bmod 13 = 10$$

$$i=0 \quad h(k, i) = 4 \bmod 13 = 4$$

$$i=1 \quad h(k, i) = (4 + 10) \bmod 13 = 1$$

$$i=2 \quad h(k, i) = (4 + 10 \times 2) \bmod 13 = 24 \bmod 13 = 11$$

iii. Insert (3):

$$h'(3) = (2 \times 3 + 7) \bmod 13 = 13 \bmod 13 = 0$$

$$h(k, i) = (0 + 0) \bmod 13 = 0 \bmod 13 = 0$$

④ Search (99) : $h'(99) = (2 \times 99 + 2) \% 13$
 $= 10$

$h(k_i) = (10 + 0) \% 13$
 $= 10$ [Blank Box]
 [Not Found]

$O(V^2)$

2. What are the benefits of the bellman ford algorithm over Dijkstra's algorithm? [2]

Bellman Ford →

- ① Can detect negative cycle
- ② works correct if neg edge exists

→ after N iteration if the value changes means there's a neg cycle

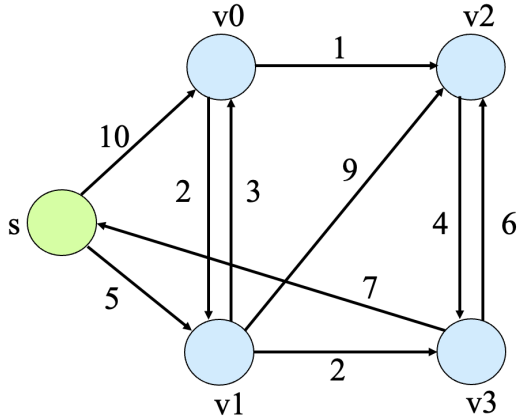
Student ID -

Name -

United International University (UIU)
Dept. of Computer Science & Engineering (CSE)
CSE 2217 Summer 2022 Class Test 4 Total Marks: 10
You have to answer here. No extra page will be provided.

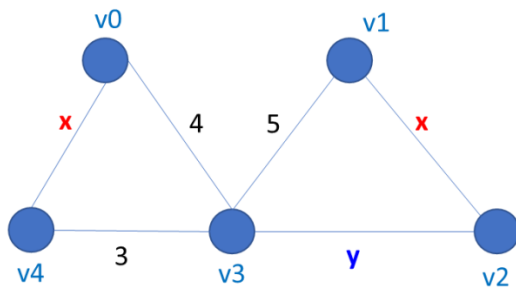
$X = \text{last_digit_of_student_id} \% 4$, $Y = X - 2 = \underline{\hspace{2cm}}$

1. Find the SPT of the given graph using Dijkstra's algorithm where the source is vertex vX. [5]



2. Find the MST of the given graph using Kruskal's algorithm. Show details calculation.

[3]



3. Analyze the runtime of the following algorithm.

[2]

```

MST-Prim( $G, w, r$ )
   $Q = V[G]$ ;
  for each  $u \in Q$ 
     $key[u] = \infty$ ;
   $key[r] = 0$ ;
   $p[r] = \text{NULL}$ ;
  while ( $Q$  not empty)
     $u = \text{ExtractMin}(Q)$ ;
    for each  $v \in \text{Adj}[u]$ 
      if ( $v \in Q$  and  $w(u, v) < key[v]$ )
         $p[v] = u$ ;
         $key[v] = w(u, v)$ ;

```

Student ID -

Name -

United International University (UIU)
Dept. of Computer Science & Engineering (CSE)
CSI 227 Summer 2022

Class Test 3 Total Marks: 10

You have to answer here. No extra page will be provided.

Use the value of x in Question 1, where $x = \text{last_three_digits_of_your_student_id} \bmod 5 =$

1 Given the parent array of a Disjoint set, perform the following operations **sequentially** using path compression and union by rank heuristic.

- a) Draw the disjoint set forest. [2]
- b) What will be returned by Find-Set(x), Find-Set(x+2) and Find-Set(7). Show details. [2]
- c) Redraw the forest after Union(7, 8). [1]
- d) Redraw the forest after Union(7, 1). [1]

Index	0	1	2	3	4	5	6	7	8	9
Parent	2	0	3	9	4	4	3	5	8	9

- 2 What are the conditions to be able to topologically sort a graph? Find the topological sort of the following graph using DFS. [1+3]

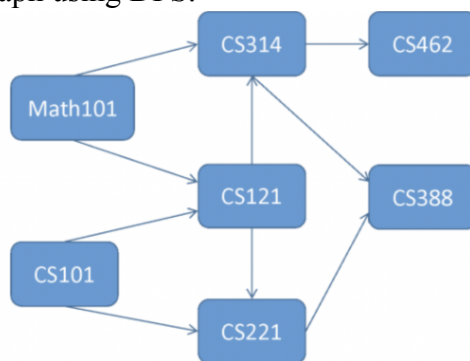
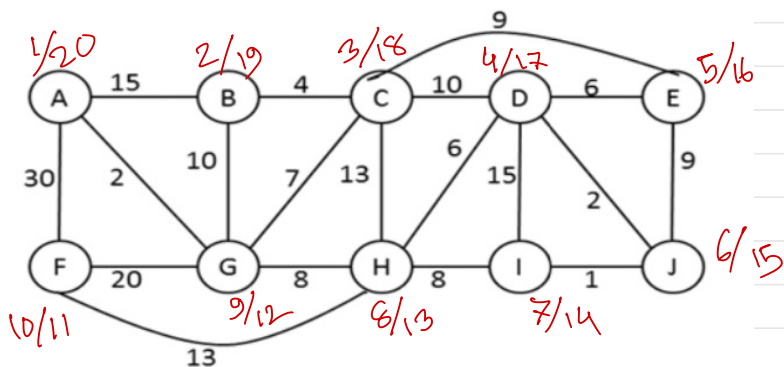


Figure 1: Course dependency graph



A
|
B
|
C
|
D
|
E
|
F

F - G - H - I - J

π

	A	B	C	D	E	F	G	H	I	J
A	A	B	C	D	E	F	G	H	I	J
	0	1	2	3	4	5	6	7	8	9